Elite JS Interview Prep Platform: Grounding System Rollout Plan

Plan only. This document does not write code, merge, deploy, or touch production data or accounts. Implementation starts in the Elite JS chat from this plan, not in this document.

Goal

Adopt the Grounding System in the Elite JS interview prep platform, using the shared concepts from `00-the-grounding-system.md`, with no stack change of any kind. No LangSmith, no Clerk, no Supabase, no new vendor. The platform keeps Next.js, Velite, Drizzle, Better Auth, and its existing providers.

The priority order is deliberate and differs from DayMirror. The platform's complexity is content, not imports. So reference verification (G2) comes first, spec and eval discipline (G4) runs through everything, the dependency graph (G1) is a small hygiene step for the codebase itself, the model-call gateway (G3) is built only if and when an AI feature ships, and tenant isolation (G5) is deferred because the platform has no multi-tenant data today.

The promise, restated for this platform: every topic, question, file, or concept an AI feature cites must exist in the platform's own content set, or the output is flagged, not shipped. Nothing inferred is ever presented as proven.

Verified current state

Everything in this section was checked for this plan. Anything not checked is marked as an assumption in the assumptions section at the end.

Repo and stack (checked: package.json, repo AGENTS.md, directory listing)

- Local checkout: `~/workspace/elite-js-interview-prep-and-learning-guide`, public repo `github.com/irfansaeedkhan/elite-js-interview-prep-and-learning-guide`.
- Stack: Next.js 16, React 19, TypeScript 5.8, Velite 0.2.2 for MDX content, Tailwind 4, Drizzle ORM with Postgres, Better Auth 1.6.23 (already in use, so Better Auth is the existing auth, not a new adoption), Vitest. Yarn only.
- Repo rules that this plan obeys: yarn only; no UI changes unless Irfan asks; business logic in services, never in controllers or components; userId on every query; content in MDX, not giant TypeScript data files; no git commits unless Irfan explicitly asks; run `yarn velite` before `tsc --noEmit` because `#site/content` is generated.
- Phase status from repo AGENTS.md: Phase 6 complete (landing plus SEO shipped). Phase 7 in progress (workflow skills, tests, codebase hardening). Site live since 2026-10-27 (from goal

memory, not re-verified here).

Content (checked: content directory, goal docs)

- Content lives in `content/tracks/` with 10 track directories: express, hono, javascript, mongodb, nestjs, nextjs, nodejs, postgresql, react, react-ecosystem.
- 65 MDX topic files counted by find. Goal docs describe the same 65 topics (40 JS, 17 React, 8 preview tracks with one topic each by design).
- Content quality process already exists and is strong: a daily research cron upgrades one weakest topic, and a daily reviewer cron independently verifies the uncommitted pile by execution. The reviewer runs code examples and prediction answers in node, breaks challenge solutions to prove tests can fail, scans frontmatter for the known YAML-colon bug, and runs the full test suite plus tsc. Verdicts per file: APPROVED, FIXED (small mechanical fixes), or REJECTED.
- Known open content item: `arrays.mdx` stays REJECTED because its "Sum with reduce" challenge tests use `console.assert`, which cannot fail. The rejection carried through 2026-10-26 and 2026-10-28. A push of the approved pile is held until the arrays fix lands and is re-reviewed (from goal memory and LEARNING.md, not re-verified in this task).
- Practical components in content: CodingChallenge, OutputPrediction, ReactPlayground, Mermaid, EventLoopExplorer. The reviewer already proves challenge tests against a faithful copy of the real harness.

Data and users (checked: src/app/api, src/db/schema, src/lib/auth)

- API routes today: `src/app/api/auth` and `src/app/api/progress` only. No AI endpoint exists.
- Tables: Better Auth tables (user, session, account, verification) plus `topicProgress`, `userStats`, and `practiceAttempts`. All progress data is per-user, scoped by userId, consistent with the repo rule "userId on every query."
- No teams, organizations, cohorts, or shared tenant data exist in the schema. No multi-tenant surface exists today.
- No AI provider code found: a grep of `src/`, `scripts/`, and `velite.config.ts` for openai, anthropic, openrouter, langchain, generative, and mistral returned nothing outside test files. No feedback endpoint exists. Assumption: no AI feature is live today. If one exists behind a flag or in an uncommitted pile, Phase 0 must surface it before Phase 3 is scoped.

Existing verification culture (checked: goal LEARNING.md)

- Case 1: a content claim about `structuredClone` was corrected from "drops functions" to "throws on functions" after execution in node. Probe: run every API behavior claim in node before writing it.
- Case 2: a challenge that cannot fail is worse than no challenge. Probe: the break-it check (a correct implementation passes, at least three deliberately broken implementations each fail at least one test).

- Case 3: a deploy failed on env vars the build validates. Probe: check the four env vars before any deploy, and verify the truncated Vercel domain, not a guessed full name.
- Standing doctrine for all projects applies here too: real-browser record plus analyze, speed gate, UI/UX screen and modal review, expert video and audio analysis, and the post-recording re-analyze and re-record loop.

Scope in

- G2 four-gate reference verification, applied first to content citations (live today, no AI needed) and then to any AI output when an AI feature ships.
- G4 spec discipline and eval discipline, adopted into crew docs and the reviewer pipeline.
- G1 dependency graph as a small hygiene script for the codebase itself, reusing the DayMirror core contract.
- G3 model-call gateway, designed and built only if and when an AI feature ships (for example, answer feedback). Designed now only to the level of a conditional spec outline, not built.
- LEARNING.md updates per phase, learning notes, and test-data cleanup with ids recorded.
- Judge gates with reproduced numbers, and live-browser Pilots on the real surface, recorded, with a separate analysis pass before anything is called done.

Scope out

- Any stack change. No LangSmith, no Clerk, no Supabase, no new vendor, no new provider. If a phase ever needs a vendor, it needs a separate explicit go from Irfan, and it is not in this plan.
- Any UI change, unless Irfan explicitly asks. G1, G2, and G3 in this plan are scripts, checkers, and server modules. If a phase later needs user-visible UI (for example, a flagged-citations view for reviewers), that UI needs its own explicit ask and its own UI/UX review.
- A visual graph canvas. The script comes first, per the system rules. A canvas is not in version 1.
- Semantic caching. Deferred in version 1 everywhere, including here.
- Quality judgments about code. The graph answers structure questions, not quality questions.
- Any git commit or push. The repo rule stands: no commits unless Irfan explicitly asks. In this plan, "accepted" means the reviewer pile accepts the work. It does not mean pushed. The held push pile and the arrays fix are unaffected by this plan except where a phase explicitly says it unblocks them.
- Building a Cardograph clone. The video's product is not the goal. Its concepts are.
- Multi-tenant features (teams, cohorts, mentor views). G5 is deferred, not built, because no such data exists today.

Components adopted

G2 reference verification: the priority

Why first: the platform's risk is wrong content and, later, wrong AI citations. A learner who memorizes a wrong interview answer pays for it in a real interview. The existing reviewer already executes code claims. G2 extends that reviewer from "does the code run" to "does every cited thing exist."

The four gates, applied to content citations (the sets are named precisely because vague sets make gates useless):

1. Evidence existence. Every cited identifier must exist in the exact set the model or author was shown. For this platform the sets are: topic slugs in the Velite-generated content set (`#site/content`, after `yarn velite` runs); question ids and interview questions inside those topics; file paths inside the repo; and route paths in `src/app`. A citation to a topic slug that does not exist fails. A citation to a file path that does not exist fails.
2. Entity allowlisting. Every cited entity name must come from the platform's own allowed list for that request. For this platform the lists are: topic names from the content set; API, method, and concept names that appear in the cited topic's own MDX or in the platform's curriculum index; and library names from the repo's own `package.json`. A concept name imported from training data that is not in the platform's content fails, even if it is a real JavaScript concept. The platform teaches its own curriculum. Citations must stay inside it.
3. Number binding. Every cited number must exactly match a canonical value surfaced from the platform's own data. For this platform the canonical values are: counts derived from the content set (topics per track, questions per topic); numbers produced by executing the cited code in node (return values, output predictions); and numbers from the user's own progress rows (attempts, scores) when output is about that user. A rounded, sign-flipped, or reformatted number fails. A performance number that was not measured fails.
4. Claim-type rules. A claim of a given type must cite evidence of the required type. A comparison claim ("X is faster than Y" or "X differs from Y") must cite both things compared, with executed or sourced evidence for each. A recommendation ("use X for this case") must cite the evidence it rests on. A behavior claim about an API must cite an executed example, per the existing Case 1 probe. A "this is asked in interviews" claim must cite the topic's own interview-question set, not a general assertion.

Behavior on failure, copied from the system doc: the output is flagged, not shipped. The checker returns the failed gate, the offending citation, and the allowed set it was checked against, so a human or a repair pass can fix it precisely. The checker never rewrites output silently. Repair is a separate, visible step: generate, check, verify, repair.

Where G2 runs, in two stages:

- Stage A (no AI needed, Phase 1): the reviewer pipeline runs the four gates over MDX content itself. Authors cite other topics, questions, and examples. Those citations are checked against the Velite content set. This catches cross-topic drift today, before any AI feature exists.
- Stage B (only when an AI feature ships, Phase 3): the same checker runs over AI output before it reaches a learner or a Judge, with allowed sets built per request from the content set plus, where relevant, that user's own progress rows. Judge scores (specificity, helpfulness, tone) are reported separately from the four deterministic gates and never override a failed gate.

G2 and the existing reviewer: G2 extends the reviewer pipeline. It does not replace it and does not create a parallel review. The break-it check, the node execution check, the frontmatter scan, the suite, and tsc all stay. The four gates are added as checks inside the same pipeline, with verdicts in the same APPROVED, FIXED, REJECTED vocabulary.

G4 spec discipline and eval discipline: runs through everything

Spec discipline, adopted into crew docs before any building:

- Every feature starts as a short spec stating what it does, what it must not do, and how success will be checked. The spec lives in the goal docs before building starts.
- If a build fails its acceptance check, discard the code, fix the spec, and rerun. Do not prompt on top of bad code. A failed acceptance means the spec was wrong.
- When ambiguous, the agent asks one specific question and brings a proposed answer (option A or B). Open-ended questions stall loops. Guesses create silent errors.
- The rules file for agents is kept short, because it is read on every prompt, and it avoids naming folders or structures the agent could then invent.
- Every spec names what is out of scope.

Eval discipline, specific to this platform:

- Interview answers and AI feedback are scored against known answers. For content, "known answer" means an executed example, a spec behavior, or a cited question with its answer in the content set. For AI feedback, "known answer" means the same, plus the learner's own attempt and progress rows.
- Prompt versions are compared by number. A current prompt and a retired prompt are scored side by side by the G2 gates and by judge scores, reported separately. A prompt change that improves a judge score but fails more gates is a regression, not an improvement.
- Judge scores are reported separately from deterministic gates, always. A high judge score never overrides a failed gate.
- The eval itself gets adversarial review. Before any eval number is trusted, someone tries to break the evaluator: a self-referencing citation, a relative-path tail match against the wrong file, a fake identifier hidden behind a name with spaces. This is the lesson from the video's eval bugs, applied here.
- Existing probes stay and are extended, not replaced: run API behavior claims in node (Case 1); break-it check for every challenge (Case 2); env-var check before deploys (Case 3).

G1 dependency graph: small hygiene step, explicitly low priority

Why low priority, stated plainly: the platform's complexity is content, not imports. The codebase is a content site with a small API surface (auth and progress). Its import graph is small, and its failure modes are content errors, not hidden coupling. Building a full graph program first would spend the crew's best week on the platform's smallest risk.

What is still worth doing, and why: shared components exist (the challenge harness, playground components, progress services), and a change to a shared component still needs an affected-area list. The G1 script answers that list mechanically instead of by grep. It also gives the affected-test selector: given changed files, the reverse walk finds every test that transitively imports them, so affected-area testing stops being guesswork.

Adoption rule: reuse the proven DayMirror core, do not rebuild it. The script emits the same contract shape (files with path, kind, line count, folder; edges with from, to, kind, resolved flag, reason when unresolved; coverage with files found, parsed, skipped with reasons, imports resolved and failed with reasons; queries for blast-radius, dependency-chain, and affected-tests at depth 2 by default). Only the repo path and the ignore list change. If the DayMirror core artifacts differ from this description when this plan executes, adapt to them. Do not rebuild a second core.

Rules carried over: an edge exists only when the parser resolves an import; every import is classified resolved, skipped, or failed with a reason; coverage is reported next to every result, always; the parser imports no web framework, UI library, or database, takes a directory path, and returns portable JSON; before-and-after parser runs on the same repo must produce identical edge lists unless the code actually changed.

G3 model-call gateway: conditional only

Trigger: an AI feature ships, for example answer feedback on practice attempts. Until that trigger, G3 is not built. Designing it now beyond a conditional outline would be speculative work.

When the trigger fires, the gateway is one server module in the existing repo (business logic in services, per repo rules) through which every model call passes. No call site talks to a provider directly.

- Tracing: every call records input, output, model, provider, token counts, latency, and cost, in the platform's own Postgres tables via Drizzle. No LangSmith. No new vendor.
- Content-keyed caching, exact-hash only: the cache key is a SHA-256 hash of everything that determines the answer (the inputs, the full system prompt, the provider, the model, and a schema version). A hit returns the cached answer with zero tokens and zero cost. Invalidation is emergent: change any input and the key changes, so the old entry is never looked up again. Nobody writes "clear cache" logic. Cache hits still emit a trace showing a cache hit, so caching is verified, not assumed. Every cached response carries metadata (hit or miss) so cost reports are honest.
- Semantic caching is deferred. It needs embeddings, a similarity threshold, and tuning, and a wrong threshold serves wrong answers confidently. Revisit only with measured demand.
- Prompt versions are stored with their schema version, so a current prompt and a retired prompt can be scored side by side by the G2 gates and by judge scores, separately.
- G2 runs on gateway output before it reaches the learner, with allowed sets built per request. A failed gate flags the output. It does not ship.

G5 tenant isolation: deferred, with the reason stated

Deferred because the platform has no multi-tenant data. All progress data is per-user, scoped by userId, and the repo already enforces userId on every query. There are no teams, clinics, companies, cohorts, or mentor views in the schema or the routes.

What already covers isolation today: userId on every query, Better Auth sessions, and the existing progress API shape. What would activate G5: a feature that introduces shared or cross-user data, for example team cohorts, mentor review of learner attempts, or shared classrooms. If such a feature is ever specced, its spec must include a G5 phase first: tenant id on every shared row, policy-first enforcement adapted to Better Auth sessions (tenant derived from the validated session, never from request payload), and the two-tenant acceptance test (two tenants, no cross-tenant rows, proven by switching tenants and reading). Until then, G5 is a named future phase, not work.

Phases in order

Phases run in the Elite JS chat, from this plan, after the plans land. No phase starts before the previous phase's Judge gate and Pilot pass, except Phase 0 documentation, which can overlap Phase 1 spec writing. No project waits for another project's phases, but no project rebuilds the core from scratch either: Phase 1 and Phase 2 adopt the DayMirror core artifacts and adapt only sets, paths, and ignore lists.

Phase 0: Foundations and spec discipline adoption

What: adopt G4 into crew docs; confirm the DayMirror core artifacts to adopt; confirm the AI-feature inventory (confirm no AI feature is live); freeze the G2 content sets and the Phase 1 spec.

Spec owner: Product Manager. Contributors: Tech Lead (core artifact confirmation), QA (reviewer pipeline mapping).

Crew assignments:

- Product Manager: owns the Phase 1 spec and the G4 adoption note in goal docs.
- Product Designer: reviews nothing user-visible in this phase (no UI changes). Confirms that no phase in this plan needs UI, or flags the UI ask if one appears.
- Frontend Engineer: maps the Velite content set shape (slugs, questions, tracks) for G2 sets. No UI work.
- Backend Engineer: confirms the DayMirror core artifacts (G1 script, G2 checker shape) and the adaptation points (repo path, ignore list, content set source).
- QA Engineer: maps the existing reviewer pipeline checks to the G2 gates, and writes the Phase 1 acceptance checks.
- Tech Lead: rules on the adaptation (what is adopted unchanged, what is adapted, what is rejected) and owns the assumption log.
- DevOps/Release: confirms the no-commit rule handling (work stays uncommitted for Irfan's review), the velite-before-tsc order, and the test command baseline.
- Engineering Manager gate: approves the Phase 1 spec and the core-adoption ruling before building starts.

- CEO agent gate: not triggered in this phase (no pricing, positioning, public surface, or money change). If a phase later touches a public surface, the CEO gate activates and new decisions escalate to Irfan.
- Client agents: junior learner (would citation checks make content more trustworthy to learn from), mid-level interview candidate (would flagged citations have saved a wrong memorized answer), mentor/reviewer (would the four gates match how a human reviewer checks citations).

Acceptance checks:

- G4 adoption note written in goal docs, in plain sentences.
- Phase 1 spec written, with what it does, what it must not do, how success will be checked, and what is out of scope.
- AI-feature inventory confirmed by grep (no provider code, no feedback endpoint) or the exception recorded with its location.
- DayMirror core artifacts named by path, or the assumption recorded that they land as `daymirror-plan.md` and will be adapted on arrival.
- Baseline numbers recorded: current test suite result, tsc result after `yarn velite`, content file count (65 at plan time), and reviewer pipeline check list.

Judge gate: Judge reproduces the baseline numbers (suite, tsc, content count) and approves the Phase 1 spec. No build is judged in this phase.

Live-browser Pilot: not applicable to a documentation phase. The Pilot obligation starts at Phase 1, where the checker runs against real content served by the real site.

Risks: the DayMirror core artifacts may land later or differ from this plan's description. Mitigation: Phase 1 adapts, never rebuilds, and the Tech Lead rules on differences. The AI inventory may miss an uncommitted pile item. Mitigation: the inventory covers the working tree, including uncommitted files, not just committed code.

LEARNING.md updates: one case or note recording the G4 adoption and the core-adoption ruling, with the assumption log referenced.

Phase 1: G2 content-citation gates in the reviewer pipeline (the priority phase)

What: implement the four gates as checks inside the existing reviewer pipeline, running over MDX content itself. Allowed sets come from the Velite-generated content set after `yarn velite` runs. Verdicts use the existing APPROVED, FIXED, REJECTED vocabulary. No UI changes. No commits unless Irfan explicitly asks.

Spec owner: Product Manager. Contributors: Backend Engineer (checker implementation), QA Engineer (gate tests and reviewer integration).

Crew assignments:

- Product Manager: owns the spec (gate definitions, set definitions, failure behavior, verdict vocabulary).
- Product Designer: no UI work. Reviews any reviewer-facing output text only if it becomes user-visible, and flags the UI ask if it does.
- Frontend Engineer: confirms the content set shape from Velite output and the component inventory (challenge, prediction, playground components) so citations to components resolve correctly.
- Backend Engineer: builds the checker as a service (business logic in services, per repo rules), with the four gates, the failure report shape (failed gate, offending citation, allowed set), and the generate, check, verify, repair flow. Reuses the DayMirror G2 checker shape where it fits.
- QA Engineer: writes gate tests, including adversarial evaluator tests (self-referencing citation, relative-path tail match against the wrong file, fake identifier hidden behind a name with spaces, a cited topic slug that does not exist, a rounded number that does not match its canonical value, a comparison claim citing only one side). Runs the existing suite plus the new gates.
- Tech Lead: owns the set definitions (what counts as the allowed list for each gate) and rules on edge cases (for example, citations to preview-track topics, citations to external specs like MDN, which are allowed only as external references, never as platform content).
- DevOps/Release: wires the gates into the reviewer pipeline order (velite, gates, execution checks, suite, tsc), keeps work uncommitted for Irfan's review, and records run ids.
- Engineering Manager gate: approves the gate definitions and the reviewer integration before the phase is judged.
- CEO agent gate: not triggered (no public surface, pricing, or money change). Reviewer output is internal.
- Client agents: junior learner (are flagged citations explained in plain language a learner can act on), mid-level interview candidate (would the gates have caught the `structuredClone` style error before it shipped), mentor/reviewer (does the failure report give exactly what a human reviewer needs to fix it).

Acceptance checks (all numbers reproduced by the Judge, not just reported):

- The checker runs over the full content set (65 topics at plan time, or the count at run time) and reports a gate result per cited reference.
- Deliberate failure fixtures: at least one planted failure per gate, each caught with the correct gate named, the offending citation quoted, and the allowed set shown.
- Adversarial evaluator tests pass: self-reference, tail-match, spaced-name, missing slug, rounded number, one-sided comparison.
- Existing reviewer checks still pass unchanged (node execution, break-it check, frontmatter scan, suite, tsc after velite).
- A previously APPROVED topic still passes (no false-positive regression), and the known REJECTED `arrays.mdx` still fails for its real reason (challenge tests that cannot fail), not for a gate artifact. If the arrays fix lands during this phase, it is judged on its own merits through the same pipeline.

- Failure reports are human-readable and actionable, and the checker never rewrites content silently.

Judge gate: Judge reproduces every acceptance number (fixture results, adversarial results, suite, tsc, content count) and gives APPROVE or the phase does not proceed. A Judge verdict of APPROVE-WITH-FIXES must name the fixes, and they must be re-verified before the phase closes.

Live-browser Pilot: the Pilot runs on the real surface (the live site or a production build served locally, per the standing rule that a surface is not done until it is opened in a real browser). The Pilot verifies that content served to a learner matches the gated content set: it opens topics whose citations were gated, confirms the cited topics and questions resolve on the live pages, and confirms no gated failure is visible to learners. The Pilot is recorded (video or frame sequence) and analyzed by a separate pass before anything is called done. If the analysis pass finds a defect, the defect is fixed and the Pilot re-runs until the analysis pass is clean.

Risks: false positives on legitimate external references (for example, MDN links) could block good content. Mitigation: external references are a separate citation class, checked for well-formedness only, never against platform sets, and the spec names this explicitly. Set staleness (Velite output older than the MDX) could produce wrong failures. Mitigation: the pipeline order is velite first, always, and the checker refuses to run on a stale generated set. Gate scope creep (checking style or tone through deterministic gates) could turn the checker into a judge. Mitigation: gates check existence, allowlisting, numbers, and claim types only. Tone and helpfulness stay in judge scores, reported separately.

LEARNING.md updates: one case per real failure found during the phase (gate bug, false positive, set definition correction), each with mechanism, fix, and probe. Plus a learning note and PDF per definition of done.

Phase 2: G1 blast-radius script for the codebase itself (hygiene, low priority)

What: adopt the DayMirror G1 script, adapted to this repo, and use it for shared-component changes and affected-test selection. No canvas. No UI. No commits unless Irfan explicitly asks.

Spec owner: Product Manager. Contributors: Backend Engineer (script adaptation), QA Engineer (affected-test verification).

Crew assignments:

- Product Manager: owns the small spec (queries, contract shape, ignore list).
- Product Designer: no work in this phase (no UI).
- Frontend Engineer: names the shared components the script must cover (challenge harness, playground components, progress UI services) and confirms their paths.
- Backend Engineer: adapts the DayMirror script (repo path, ignore list for generated content and node_modules), verifies the contract shape matches, and wires the affected-test query.
- QA Engineer: verifies the script against known cases (a shared component change lists the right affected files and tests; a parser change produces identical edge lists on unchanged

code).
- Tech Lead: rules on the ignore list and on any edge-kind question (for example, dynamic imports in playground code).
- DevOps/Release: records the script's run command and baseline coverage numbers, keeps work uncommitted.
- Engineering Manager gate: approves the script and its coverage baseline.
- CEO agent gate: not triggered (internal tooling, no public surface or money change).
- Client agents: not central in this phase (internal tooling). The mentor/reviewer agent confirms the affected-area list matches how a human reviewer scopes a shared-component change. The learner agents are not polled for an internal script.

Acceptance checks:

- The script runs from a plain command against the repo directory and emits the contract shape (files, edges, coverage, queries).
- Coverage reported next to every result (files found, parsed, skipped with reasons, imports resolved and failed with reasons).
- Known-case verification: blast radius for the challenge harness lists the topics and tests that import it; dependency chain for a progress service lists what it relies on.
- Regression rule verified: two runs on unchanged code produce identical edge lists.
- Affected-test query verified: given a changed shared file, the reverse walk finds the tests that transitively import it, and running only those tests plus the full suite gives the same verdict on a planted failure.
- The script imports no web framework, UI library, or database.

Judge gate: Judge reproduces the coverage numbers, the known-case lists, and the regression check, and gives APPROVE or the phase does not proceed.

Live-browser Pilot: minimal but real. The Pilot is not a UI feature, so the browser check verifies the script's output against the live site's structure: it opens pages that use a shared component named in a blast-radius result and confirms the component is actually present on those pages. Recorded, with a separate analysis pass. If the analysis pass finds a mismatch between the graph and the live pages, the graph or the spec is fixed and the Pilot re-runs.

Risks: the script may be used rarely, because the codebase is small. Mitigation: that is acceptable. The phase is deliberately small. If usage stays low after two shared-component changes, the script stays as a tool, and no further G1 work is scheduled. Parser edge cases in playground code (dynamic imports, Sandpack code strings) may produce unresolved imports. Mitigation: unresolved imports carry reasons, and code strings inside Sandpack are not repo imports, so the spec names them as out of graph scope.

LEARNING.md updates: one note recording the baseline coverage numbers and any parser edge case found, with mechanism and probe.

Phase 3: G3 model-call gateway, only if and when an AI feature ships (conditional)

Trigger, stated as a gate, not a date: this phase starts only when a spec for a real AI feature (for example, answer feedback on practice attempts) is approved. If no AI feature is specced, this phase does not start, and nothing in this plan is blocked by its absence. The G2 Stage B checker (same gates, applied to AI output) is part of this phase, not a separate phase.

Spec owner: Product Manager, with the AI feature's own spec as the parent spec. Contributors: Backend Engineer (gateway module), QA Engineer (trace and cache verification).

Crew assignments:

- Product Manager: owns the AI feature spec and the gateway spec (call sites, trace shape, cache key composition, schema versioning).
- Product Designer: UI work only if the AI feature itself has user-visible UI and Irfan explicitly asks for it. If so, full UI/UX review through the skills with local-ui-ux-director sign-off before clearance.
- Frontend Engineer: builds only what the AI feature spec names. No gateway UI unless specced.
- Backend Engineer: builds the gateway as one server module (business logic in services), with tracing to the platform's own Postgres tables via Drizzle, exact-hash SHA-256 caching, cache-hit traces, and hit-or-miss metadata on every cached response. No provider call site bypasses the gateway.
- QA Engineer: verifies tracing completeness (every call traced, cache hits traced as hits), cache correctness (identical inputs produce identical keys and identical answers; any input change produces a different key and a fresh call), and G2 Stage B gating on gateway output (planted invented references caught, judge scores reported separately).
- Tech Lead: owns the cache key composition (inputs, full system prompt, provider, model, schema version) and rules on what counts as an answer-determining input.
- DevOps/Release: confirms env handling (provider keys in env only, validated in `src/lib/env.ts`, never in code), keeps work uncommitted unless Irfan asks, and records cost baselines from traces.
- Engineering Manager gate: approves the gateway spec and the AI feature spec together, since the gateway is meaningless without a real call site.
- CEO agent gate: activates if the AI feature touches a public surface, positioning, or money (for example, a paid feedback tier). New pricing or positioning decisions escalate to Irfan. If the feature is free and internal to existing pages, the CEO gate records a pass with no decision needed.
- Client agents: junior learner (is flagged feedback explained in plain language, and is a flagged answer better than a confident wrong answer), mid-level interview candidate (would the feedback have helped a real interview answer, or does it add noise), mentor/reviewer (does the feedback match what a human mentor would say, and does the trace let a mentor audit it).

Acceptance checks:

- Every model call in the AI feature passes through the gateway. A grep for direct provider calls outside the gateway returns nothing.

- Tracing: every call has a trace row with input, output, model, provider, token counts, latency, and cost. Cache hits have trace rows marked as hits with zero tokens.
- Caching: identical inputs return the cached answer with zero new tokens. Any change to inputs, system prompt, provider, model, or schema version produces a fresh call. No "clear cache" logic exists.
- G2 Stage B: planted invented topic slugs, question ids, and numbers in AI output are caught by the correct gate, with the failure report naming the gate, the citation, and the allowed set. Judge scores are reported separately and never override a failed gate.
- Cost baseline: trace data produces a cost-per-feature report for the AI feature's first real usage window.
- No semantic caching exists. A test confirms that a reworded question does not reuse a cached answer.

Judge gate: Judge reproduces the trace counts, the cache hit and miss numbers, the G2 Stage B fixture results, and the cost report, and gives APPROVE or the phase does not proceed.

Live-browser Pilot: the Pilot runs the AI feature on the real surface with a real learner account (demo, staging, or throwaway only, never production learner data unless a later phase explicitly approves it). It records the full flow (attempt, feedback, flagged case if any), and a separate analysis pass checks the recording for correctness, timing, and UI defects before anything is called done. If the analysis pass finds a defect, fix and re-record until the analysis pass is clean.

Risks: provider costs could grow with usage. Mitigation: tracing makes cost visible from day one, and exact-hash caching removes repeat costs for unchanged content. A gateway bug could block all AI calls. Mitigation: the gateway is a thin module with its own tests, and a gateway failure must fail loudly, never silently serve a stale or wrong answer. Judge-score drift could be mistaken for quality change. Mitigation: judge scores are reported separately from gates, and prompt comparisons use gate numbers as the primary verdict.

LEARNING.md updates: one case per real failure (gateway bug, cache key miss, gate miss on AI output), each with mechanism, fix, and probe. Plus a learning note and PDF per definition of done.

Phase 4: G5 tenant isolation, only if multi-user shared data ships (conditional, deferred)

Trigger: a spec for shared or cross-user data (team cohorts, mentor review of learner attempts, shared classrooms). Until that trigger, this phase does not exist as work. It is named here so the trigger is recognized when it appears.

If triggered, the spec owner is the Product Manager, with the feature spec as the parent spec. The phase content, in brief: tenant id on every shared row; policy-first enforcement adapted to Better Auth sessions (tenant derived from the validated session, never from request payload); the two-tenant acceptance test (two tenants, no cross-tenant rows, proven by switching tenants and reading); G2 allowed sets built per tenant, per request; scoped credentials wherever an agent or background job acts across a tenant boundary; service keys and admin paths kept off clients with their own audit trail.

Judge gate and Pilot for this phase, if triggered: Judge reproduces the two-tenant test numbers and the cross-tenant probe results (a tampered tenant id reads nothing, a credential for another tenant reads nothing). The Pilot runs on the real surface with two throwaway tenant accounts, recorded, with a separate analysis pass. Nothing in this phase touches real learner data without its own explicit approval.

LEARNING.md updates: cases for any isolation failure found, with mechanism, fix, and probe.

Risks across the whole rollout

- Set staleness: G2 sets come from Velite output. If Velite output is older than the MDX, gates fail wrongly. Mitigation: pipeline order is velite first, always, and the checker refuses a stale generated set.
- False positives on external references: MDN links and spec links are not platform content. Mitigation: external references are a separate citation class, checked for well-formedness only.
- Gate scope creep: deterministic gates must not become style or tone judges. Mitigation: gates check existence, allowlisting, numbers, and claim types only. Tone stays in judge scores, reported separately.
- The DayMirror core may land later or differ. Mitigation: adapt, never rebuild. The Tech Lead rules on differences.
- The no-commit rule may slow landing. Mitigation: that rule is Irfan's, and it stands. Accepted work waits in the pile until he asks for a push. The plan does not work around it.
- An AI feature may never ship. Mitigation: Phase 3 is conditional, and Phases 1 and 2 deliver value without it. Nothing in this plan depends on AI existing.
- Learner trust risk in reverse: over-flagging could make content feel uncertain. Mitigation: flags go to reviewers, not to learners, until a phase explicitly specs learner-visible flagging with its own UI ask and review.

Definition of done (per phase, and for the rollout)

A phase is done only when all of these hold:

- Built to its spec, with the spec updated to match what actually shipped.
- Guardian tests pass, including real-database checks where the phase touches data, and security probes where it touches tenants or credentials (Phase 3 and the conditional Phase 4).
- Judge gives APPROVE with numbers reproduced, not just reported.
- Pilot passes on a live browser on the real user-facing surface, recorded, and analyzed by a separate pass that comes back clean. Test suites, database writes, and agent completion reports alone are claims, not proof.
- LEARNING.md updated with real cases, plus a learning note and PDF per definition of done.
- Test data cleaned up, with ids recorded.
- Work stays uncommitted unless Irfan explicitly asks for a commit or push. "Done" never means "pushed" in this repo unless he says so.

The rollout is done when Phase 1 and Phase 2 are done and Phase 3 and Phase 4 are either done or explicitly recorded as not triggered. A conditional phase that never triggers is a completed decision, not an open item, provided the trigger and the reason are written in the goal docs.

Assumptions (unverified, never stated as fact above)

- The DayMirror core artifacts (G1 script, G2 checker shape) are assumed to land as described in `00-the-grounding-system.md` and in a DayMirror plan. This plan did not re-verify a DayMirror plan file, because only the system doc existed in the grounding-system directory at plan time. If the core differs, Phase 0 adapts to it.
- No AI feature is live today. This rests on a code grep, not on a full audit of uncommitted piles, flags, or drafts. Phase 0 re-confirms the inventory against the working tree.
- The live site state, the current branch and commit state of the repo, and whether the daily research and reviewer crons are still running were not re-verified for this plan. Phase 0 records them as baselines.
- Content counts (65 topics, 10 tracks) are as counted at plan time. They will drift as daily topic work continues. Phases record the count at run time, not plan time.
- The arrays fix and the held push pile are as described in goal memory and LEARNING.md. Phase 1 judges the arrays file on its own merits through the same pipeline and does not assume its state.

Handoff

Implementation starts in the Elite JS chat from this plan, not in this document. The Elite JS crew takes Phase 0 first: adopt G4, confirm the core artifacts, confirm the AI inventory, freeze the Phase 1 spec, and record baselines. No code is written from this plan outside that chat, and no phase here authorizes a commit, a push, a deploy, or any touch to production learner data.